ARGMAX MINI

ARCHITECTURE CASE STUDY / 2026

AWS EKS-based MLOps Platform Architecture

Designing a resilient ML platform for bursty GPU training, isolated data processing, and always-on inference.

A source-grounded case study focused on workload boundaries, elastic capacity, failure containment, durable asynchronous workflows, and platform operability.

TARGET ARCHITECTURE + LIMITED API IMPLEMENTATION

ROLE

System Architecture & Backend Design

Designed the target architecture and reliability model; implemented the limited API/data foundation.

DevOps / SRE / Infrastructure Engineering Portfolio

AWS EKS target architecture

Designed, not represented as production operation

11 pages | selectable text | vector-first diagrams

01 / WORKLOAD CHARACTERISTICS

One platform, four different operating models

The architecture starts with lifecycle, resource profile, failure domain, and scaling behavior - not with a favorite service.

ALWAYS-ON

Backend API

LIFECYCLE

short request / response

COMPUTE

CPU | control-plane work

ISOLATION

API Pod

SCALING

replicated Pods on managed nodes

K8S JOB

Dataset processing

LIFECYCLE

up to 2 GB per version

COMPUTE

CPU / memory intensive

ISOLATION

one Job per DatasetVersion

SCALING

controller creates Jobs

K8S JOB

GPU training

LIFECYCLE

12-24 hour batch

COMPUTE

bursty, scarce GPU

ISOLATION

one Job per TrainingJob

SCALING

Karpenter GPU capacity

ALWAYS-ON

Model serving

LIFECYCLE

latency-sensitive traffic

COMPUTE

CPU serving for XGBoost

ISOLATION

KServe InferenceService

SCALING

minimum replicas >= 1

ARCHITECTURE CONSEQUENCE

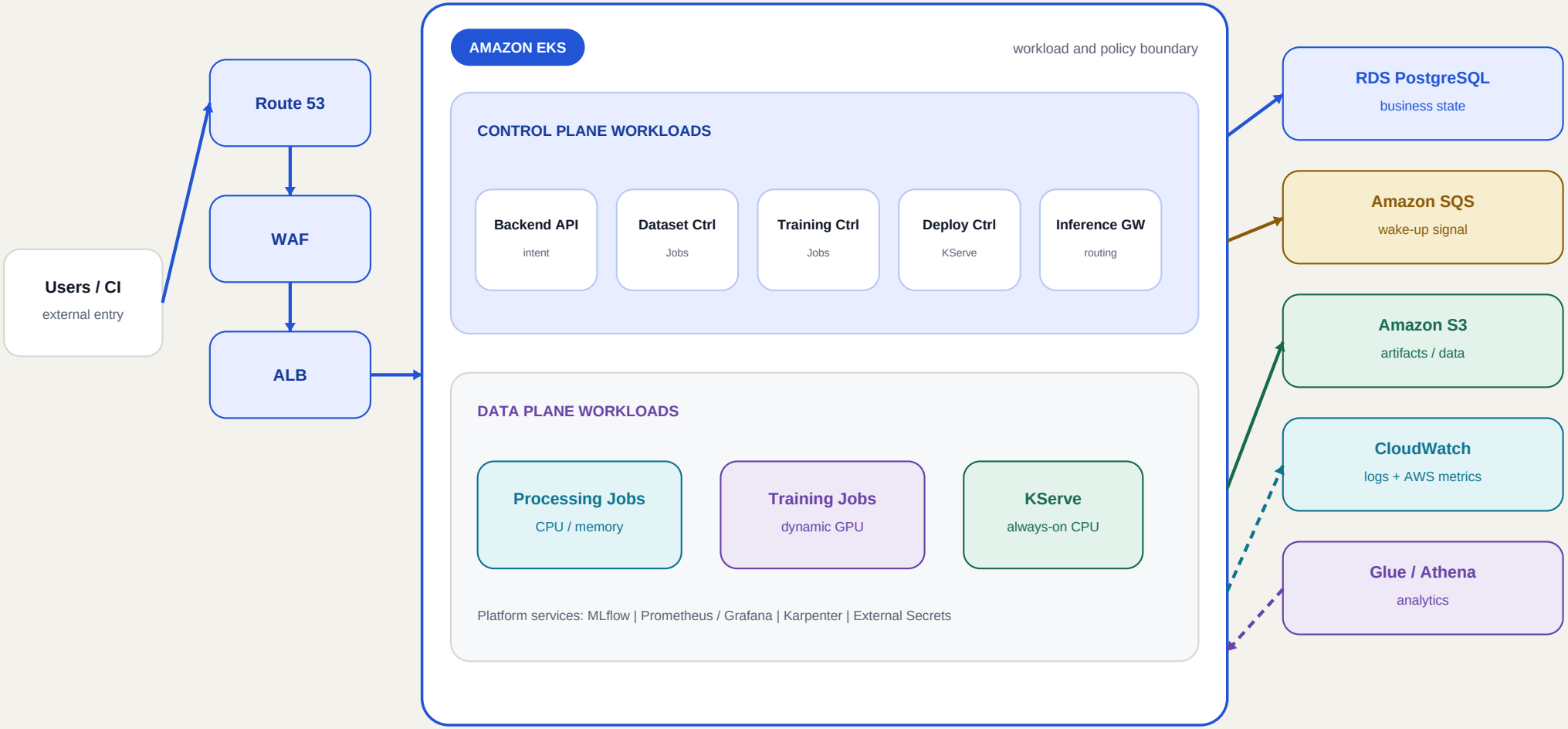
Separate execution boundaries keep a 2 GB parse or a 24-hour GPU run out of the API request path.

02 / CONTROL PLANE + DATA PLANE

TARGET ARCHITECTURE / DESIGNED

Durable intent in PostgreSQL; isolated execution in Kubernetes

Simplified vector redraw of the repository system architecture diagram. Managed AWS services remain outside the EKS execution boundary.



- **RDS** — source of truth
- **SQS** — at-least-once signal
- **K8s API** — execution observation
- **S3** — large-object path

03 / KUBERNETES CAPACITY

TARGET ARCHITECTURE / DESIGNED

Keep the platform warm; make expensive GPU capacity elastic

Karpenter is scoped to GPU training. Core services stay on a managed node group, and GPU nodes may scale to zero when no training Pod is pending.

MANAGED NODE GROUP

Always-on platform baseline

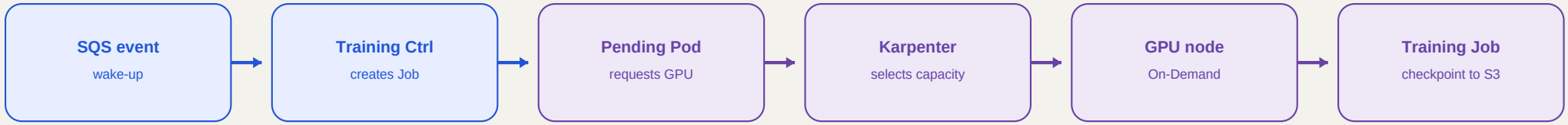
- Backend API, controllers, Inference Gateway and KServe baseline
- MLflow, observability, Karpenter Controller, CoreDNS and cluster services
- Stable capacity for latency-sensitive and control-plane workloads

KARPENTER GPU NODEPOOL

Burst capacity for training only

- Provisioned by a pending Pod requesting nvidia.com/gpu
- GPU labels, taints, tolerations and affinity prevent accidental placement
- NodePool limits cap capacity; On-Demand is the default capacity type

ELASTIC GPU PROVISIONING LOOP



Cost controls bound elastic capacity (target design)

- GPU nodes scale to zero when idle
- NodePool limit caps aggregate GPUs
- On-Demand is the reliability baseline
- Spot only after checkpoint / retry validation
- Consolidate empty nodes; protect long jobs
- KServe Standard accepts warm-serving cost

04 / FAILURE ISOLATION

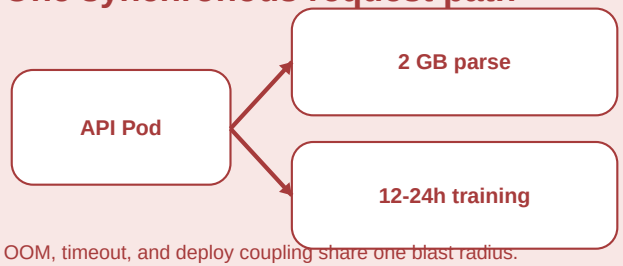
TARGET ARCHITECTURE / DESIGNED

Design the blast radius before designing the happy path

Large uploads and long-running training are execution workflows, not API request handlers. Each unit of work gets its own failure boundary.

AVOID

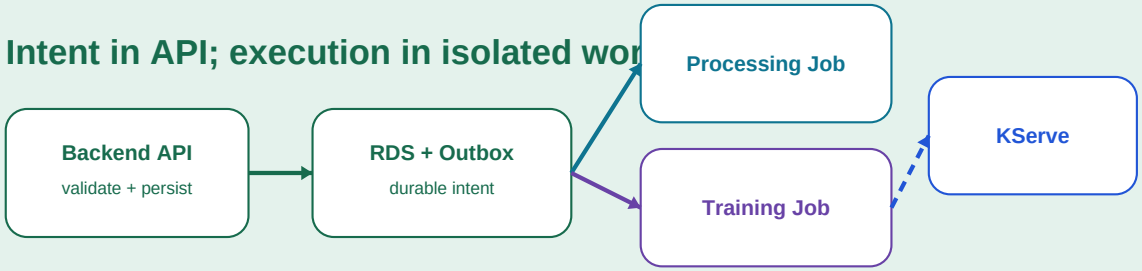
One synchronous request path



OOM, timeout, and deploy coupling share one blast radius.

DESIGNED

Intent in API; execution in isolated wor



Each resource is observable, retryable, and replaceable on its own boundary.

FAILURE DOMAIN MATRIX

Processing runtime failure

BLAST RADIUS

one DatasetVersion Job

RECOVERY MECHANISM

retry only if classified retryable

GPU / node loss

BLAST RADIUS

one TrainingJob

RECOVERY MECHANISM

checkpoint + retry

Duplicate SQS

BLAST RADIUS

one event delivery

RECOVERY MECHANISM

idempotency + conditional update

API rollout

BLAST RADIUS

API Deployment

RECOVERY MECHANISM

training continues independently

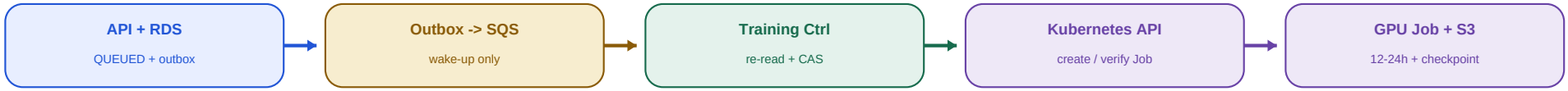
05 / RELIABILITY MECHANICS

TARGET ARCHITECTURE / DESIGNED

Separate the queue handoff from the 12-24 hour training lifecycle

SQS wakes the controller; it is not the job store. PostgreSQL owns business intent, while Kubernetes reports execution state and S3 holds checkpoints.

SQS HANDOFF VS LONG-RUNNING EXECUTION

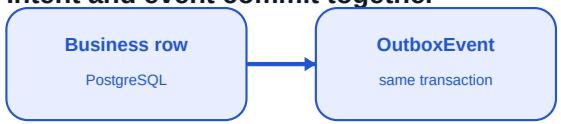


QUEUE HANDOFF / RETRY WINDOW

EXECUTION LIFECYCLE

1 / OUTBOX + RETENTION

Intent and event commit together



- Published rows: retain 30 days, then cleanup job
- Unpublished rows: never auto-delete
- Retry with backoff; duplicate delivery remains possible

2 / TRAININGJOB STATE

Advance only from a valid predecessor

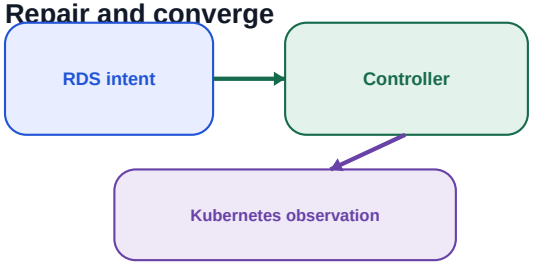
FAILURE active state -> FAILED

CANCEL RACE CANCEL_REQUESTED -> CANCELLED / SUCCEEDED / FAILED

Terminal observation decides the final business state.

3 / 60s RECONCILER

Repair and converge



Not an event-latency or user-facing SLO.

Shorter = more polling | Longer = more recovery lag

CONDITIONAL TRANSITION

```
UPDATE training_jobs SET status='RUNNING' WHERE id=:id AND status='SCHEDULING';
```

Only rows_affected = 1 advances; deterministic Job names absorb duplicate effects.

06 / OBSERVABILITY

TARGET ARCHITECTURE / DESIGNED

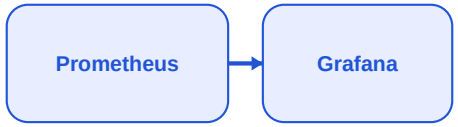
Correlate user intent, controller decisions, Kubernetes execution, and AWS signals

Metrics, logs, and identifiers are designed around a distributed workflow. UUIDs belong in structured logs, not high-cardinality Prometheus labels.

PROMETHEUS STACK

Platform and workload metrics

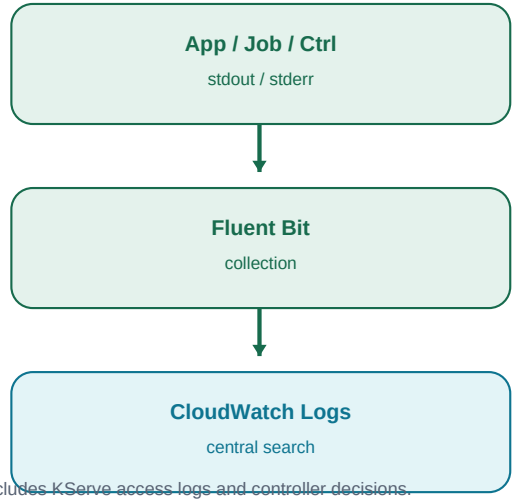
- Backend API and controllers
- Processing / Training Jobs and KServe
- Karpenter and Kubernetes Nodes
- kube-state-metrics + node-exporter
- NVIDIA DCGM Exporter for GPU signals



Alertmanager routes actionable platform alerts.

STRUCTURED LOG PIPELINE

One path from Pod to search



Includes KServe access logs and controller decisions.

AWS SERVICE SIGNALS

CloudWatch service context

CORRELATION CONTRACT

Synchronous	request_id	**Asynchronous**	event_id + resource_id	**Structured fields**	service | user_id | training_job_id | error_code

Identifiers live in logs for traceability; bounded dimensions live in metrics for operational aggregation.

07 / SECURITY

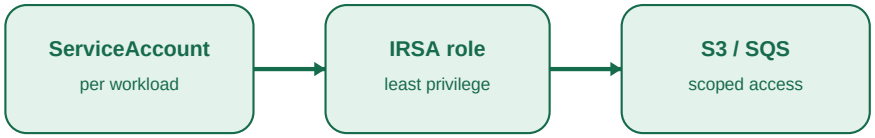
TARGET ARCHITECTURE / DESIGNED

Identity and trust boundaries follow the workload, not the node

External traffic is filtered before EKS, AWS access uses workload identity, and secrets are synchronized without long-lived access keys in Pods.

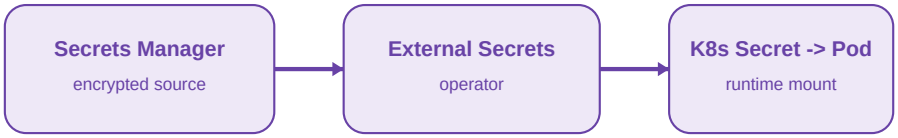
EXTERNAL REQUEST PATH

AWS WORKLOAD IDENTITY



Pods do not store long-lived AWS access keys.

SECRET DELIVERY



Only the operator reads Secrets Manager directly.

DEFENSE-IN-DEPTH CONTROLS

NETWORK

Security Groups + VPC CNI NetworkPolicy; default-deny namespaces

POD

non-root | no privilege escalation | drop all capabilities | RuntimeDefault seccomp

ENCRYPTION

external TLS | S3 SSE-KMS | RDS / SQS / EBS encryption at rest

PRIVATE

RDS is not public; KServe and MLflow remain cluster-private

08 / CI/CD + GITOPS

TARGET ARCHITECTURE / DESIGNED

A Git contract turns deployment into a reconciled platform capability

Target Architecture - Designed, Not Deployed. Developers change code and declared configuration; the delivery system converges the cluster.

DESIGNED DELIVERY FLOW

CI validates and produces an artifact. Git records desired deployment state. Argo CD owns cluster convergence.

CONTINUOUS INTEGRATION

- run tests and lint checks
- build container image and vulnerability scan
- push immutable artifact to Amazon ECR
- update declared image tag / Helm values

CONTINUOUS DELIVERY

- Argo CD watches the declared Git state
- Helm renders repeatable Kubernetes resources
- drift is detected and reconciled through one control loop
- cluster changes remain auditable through the Git history

DEVELOPER PLATFORM ANGLE

Developers interact with a reviewed Git contract - not ad hoc cluster operations.

Terraform is intentionally outside this documented delivery scope.

09 / TRADE-OFFS

TARGET ARCHITECTURE / DESIGNED

Add complexity only when the workload pays for it

The platform accepts operational costs where they buy isolation, durability, or elastic scarce capacity - and postpones components without a demonstrated requirement.

ADOPTED IN THE TARGET DESIGN

Amazon EKS

One scheduler and policy model for Jobs, GPU, and KServe.

COST: CLUSTER OPERATIONS

SQS + Outbox

Durable async intent without coupling work to an API request.

COST: IDEMPOTENCY + PUBLISHER

Karpenter

Elastic GPU nodes for bursty 12-24h training workloads.

COST: SCHEDULING LATENCY

PostgreSQL

Constraints and transactions provide the source of truth.

COST: RECONCILIATION DUTY

KServe Standard

Standard cluster-local serving without a Knative dependency.

EFFECT: MIN REPLICAS >= 1

NOT ADOPTED INITIALLY

Redis

No demonstrated metadata bottleneck beyond PostgreSQL.

REVISIT WITH EVIDENCE

KEDA

The controller already consumes SQS and creates Kubernetes Jobs.

AVOID DUPLICATE CONTROL LOOP

Knative

No scale-to-zero or event-streaming requirement for initial serving.

KEEP SERVING SIMPLER

Envoy Gateway

ALB + Inference Gateway + ClusterIP cover documented routing.

AVOID ANOTHER GATEWAY

CloudFront

No static distribution or download-acceleration requirement.

ADD ONLY IF WORKLOAD CHANGES

10 / EVIDENCE + PORTFOLIO LINKS

Designed architecture, implemented foundations, explicit boundary

The case study shows system-level reasoning without representing the AWS target architecture as already deployed or operated in production.

DESIGNED / TARGET ARCHITECTURE

AWS EKS platform system

- RDS, S3, SQS and EKS service boundaries
- workload-isolated processing, training, and serving
- Karpenter dynamic GPU NodePool and guardrails
- Outbox, idempotency, conditional updates, reconciliation
- KServe, MLflow, observability, security, CI/CD and GitOps

IMPLEMENTED / REPOSITORY EVIDENCE

Limited API and data foundation

- FastAPI Dataset API: create, detail, update, delete + health
- PostgreSQL schema with 14 tables and Alembic migrations 0001-0004
- transactional audit logging, soft delete, aggregate query behavior
- Docker Compose development path and automated tests
- No claim of deployed EKS, Karpenter, KServe, MLflow, or GitOps

DECISION LINEAGE

Workload facts → Architecture decision → Failure mechanics → Operational signals → Evidence boundary

LIVE ARCHITECTURE PORTFOLIO

https://naladoodong.github.io/Kubernetes-EKS-MLOps/

SOURCE REPOSITORY

https://github.com/naladoodong/Kubernetes-EKS-MLOps